

Figure 4: Structure of a hybrid kernel-based operating system

Since a micro-kernel's size is lower compared to monolithic kernels, and the kernel is designed for a dedicated system, we can achieve predictability (or worst-case response time) for time-critical tasks. Examples of micro-kernel-based operating systems are Nucleus, eCos, QNX and WinCE. In contrast to our above discussion, we have the VxWorks RTOS, whose kernel is monolithic.

Hybrid kernels

Due to the increasing complexity of embedded devices, at times we need to execute both hard real-time applications and general-purpose applications on a single system. In such a case, both GPOSs and RTOSs are unsuitable: with a GPOS, we cannot get predictable behaviour for the real-time applications, and with an RTOS, the general-purpose applications will considerably degrade system performance. To cater for such a contrasting requirement, we need a hybrid kernel, which offers both real-time and general-purpose features. Real-time applications execute directly in the real-time kernel, and when there are no real-time tasks running, then non-real time tasks are allowed to run. When a real-time task wants to access a certain feature that is not available in the micro-kernel, it exploits a feature from the monolithic kernel via a special communication mechanism using FIFO. Real-Time Application Interface (RTAI), Real-Time Linux (RT Linux) and Windows NT [2] are examples of hybrid kernel-based operating systems. Figure 4 depicts the structure of a hybrid kernel-based operating system.

We can choose an appropriate operating system once we identify our target system as one of the following:

- Desktop or server
- Embedded system
- Embedded system with soft real-time requirements
- Embedded system with hard real-time requirements

Figure 5 is a block diagram with examples of different target systems classified into GPOSs, soft real-time and hard real-time targets.

In the embedded world of consumer electronics (CE) or telecommunications, we should follow standards that specify

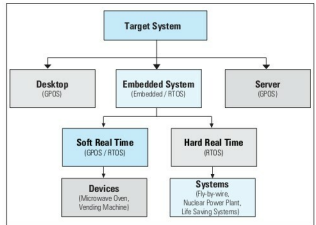


Figure 5: Block diagram of different target systems

the necessary features of the system, such as Consumer Electronic Linux Forum (CELF) specifications, or Carrier Grade Linux (CGL) Specifications [3,4].

An architect's ideal design goal

The ideal design goal of the architect is to create a system or solution with acceptable levels of reliability and performance for the given product and customer. Architects should also check if they need to provide a reasonable Time To Market.

Reliability

Reliability is the continuous assurance of the integrity and consistency of the system and all its transactions. It is defined as the probability that a device will perform its required function under stated conditions for a specific period of time. The ability to predict this with some degree of confidence depends on the correct definition of a number of parameters. Good reliability can only be achieved by sound engineering design and working towards perfection in all aspects of the product life cycle. Product reliability is seen as a proof of the robustness of the hardware, platform and application design.

Software reliability is inversely proportional to the number of defects per KLOC (Kilo Lines Of Code) executed on the system, and the reliability of the product is proportional to the MTBF (Mean Time Between Failures).

Performance

Performance means different things in different system scenarios. Performance is a measure of the effectiveness of your application in terms of response time, transaction throughput, and/or resource usage. It involves trade-offs between cost and benefits.

Time To Market

Time To Market (TTM) is the time from when a product or system is conceived, to the time that it is available for sale. Use of Commercial Off-The-Shelf (COTS) products is encouraged, to achieve better time to market. TTM is crucial